

Strided Frobenius Additive FFT and its Application to HQC

Ming-Shing Chen¹, Tun-You Chien^{1,2}, Chun-Ming Chiu¹, Cesare Huang¹
Han-Hsuan Lin², Chun-Tao Peng¹ and Bo-Yin Yang¹

¹ IIS and CITI, Academia Sinica, Taipei, Taiwan

{mschen, by}@crypto.tw, r12922085@csie.ntu.edu.tw, cesarehuang@icloud.com,
PengTaiwan6517@gmail.com

² National Tsing Hua University, Hsinchu, Taiwan

daniel0702chien@gmail.com, linhh@cs.nthu.edu.tw

Abstract. Boolean polynomial multiplication is the primary computational bottleneck of the Hamming Quasi-Cyclic (HQC) key encapsulation mechanism. In this paper, we reframe the Frobenius Additive FFT (FAFFT) in ring-theoretic terms, via quotienting homomorphisms and the Chinese Remainder Theorem. This perspective shows that a complete decomposition into evaluation points is unnecessary for multiplication, and naturally yields the Strided FAFFT (SFAFFT), which operates over smaller finite fields with fewer butterfly stages and admits a much sparser CRT modulus for the non-power-of-two degrees in HQC. Our SFAFFT implementations outperform all previous FAFFT-based multiplications on every tested platform (x86 AVX2, GFNI, Apple M1, ARM Cortex-A72, and Cortex-M4), and set new overall speed records for HQC in nearly all settings except plain AVX2, where Toom–Cook–Karatsuba remains faster for the two smaller parameter sets.

Keywords: HQC · additive FFT · Boolean polynomial multiplication · Implementation

1 Introduction

The Frobenius Additive Fast Fourier Transform (FAFFT) [LCK⁺18] has been an essential algorithm to accelerate large-scale Boolean polynomial multiplications, particularly within code-based post-quantum cryptographic schemes like HQC [AAB⁺25]. However, despite its empirical adoption, integrating FAFFT into practical cryptographic implementations reveals several critical gaps that remain unresolved in prior literature.

First, the mathematical formalization of FAFFT in existing publications often lacks rigorous structure. The underlying proofs of correctness are frequently presented through complex, case-specific computations, obscuring the algebraic principles behind the technique. This lack of a unified theoretical framework hinders further algorithmic generalizations and optimizations. Second, while previous works [CCC⁺26, JLK⁺25] have recognized the benefit of using sparse moduli for the CRT mappings in HQC, they have not reached the optimal level of sparsity. They overlook that modifying the underlying FFT structure can generate an even sparser modulus, transforming what is currently an optimized mapping into a significantly lighter operation. Consequently, there remains significant room for optimization to achieve better throughput and instruction-level efficiency across diverse hardware architectures.

To address these theoretical and practical limitations, this paper fundamentally reframes the understanding of additive FFTs [Can89, GM10, LCH14]. Traditionally, these algorithms have been viewed primarily as fast evaluation-interpolation strategies. We propose a paradigm shift by adopting a rigorous ring-theoretic perspective. By formalizing the FAFFT through quotient-ring homomorphisms and the Chinese Remainder Theorem, we demonstrate that a complete decomposition into individual evaluation points is not strictly required for polynomial multiplication. Because multiplication in the quotient ring recovers the product exactly as long as the degree of the quotient algebra prevents wraparound, the algorithmic requirements can be significantly relaxed. This structural insight unifies the mathematical framework and paves the way for our proposed optimizations, allowing us to introduce strided and incomplete transformations that map far more efficiently onto modern hardware architectures.

Our Contributions Our contributions are threefold:

Ring-theoretic formalization of FAFFT. We reframe the traditional evaluation-centric view of FAFFT into a structured algebraic foundation. By interpreting the transform through quotient-ring homomorphisms and the Chinese Remainder Theorem, we replace complex, case-specific computations with a unified theoretical framework.

Derivation of Strided-FAFFT (SFAFFT). Leveraging our ring-theoretic perspective, we present the Strided Frobenius Additive FFT (SFAFFT). We demonstrate that by focusing on the computation algebra rather than complete evaluation decomposition, introducing stride lengths and incomplete transforms becomes a natural and mathematically straightforward extension.

High-performance cryptographic implementations. We accelerate the polynomial multiplication in the HQC scheme by utilizing SFAFFT. Our implementations outperform all previous FAFFT-based multiplications on every tested platform—high-end CPUs (x86 AVX2, GFNI, Apple M1), edge devices (ARM Cortex-A72), and resource-constrained embedded systems (ARM Cortex-M4)—and set new overall speed records for HQC on most of them.

Related Work Additive Fast Fourier Transform (FFT) is an efficient technique for evaluating polynomials over additive subgroups of a coefficient field. Cantor [Can89] first established the foundational concepts and computational framework for this approach. Subsequently, Gao and Mateer [GM10] introduced new additive FFT algorithms based on Taylor expansions over finite fields of characteristic two, focusing primarily on the efficiency of polynomial evaluation. Lin et al. [LCH14, LANHC16] further accelerated the algorithm by introducing novel polynomial bases. Traditionally, these foundational works view additive FFT primarily as a fast evaluation-interpolation strategy. In this paper, we aim to reframe this evaluation-centric approach into a ring-theoretic perspective, providing a more structured algebraic foundation for polynomial multiplication.

To further optimize polynomial arithmetic over finite fields, Van der Hoeven and Larrieu [vdHL17] leveraged the Frobenius map to accelerate traditional multiplicative FFT computations. Inspired by this structural symmetry, Li et al. [LCK⁺18] and Chen et al. [CCK⁺18] applied the Frobenius map to additive FFT, formally introducing the Frobenius Additive Fast Fourier Transform (FAFFT) to achieve efficient polynomial multiplications over $\mathbb{F}_2[x]$.

In the context of cryptography, FAFFT has been widely adopted to optimize large polynomial arithmetic. For instance, Chen et al. [CCK21] integrated FAFFT to accelerate the BIKE scheme, while Lee et al. [LJKH25] utilized it to implement the HQC scheme. Ras et al. [RLC⁺25a, RLC⁺25b] presented optimizations on a RISC-V-based System-on-Chip.

To handle the specific polynomial dimensions required by HQC [AAB⁺25], which do not align with the dimensions ideal for additive FFT, recent implementations have combined additive FFT with the Chinese Remainder Theorem (CRT). Chen et al. [CCPY26] first proposed decomposing the polynomial multiplication via CRT and accelerating the sub-components with additive FFT, utilizing Single Instruction Multiple Data (SIMD) instructions on high-end CPUs. Building upon this, recent works [CCC⁺26, JLK⁺25] mapped and optimized this CRT-based additive FFT multiplication strategy onto the resource-constrained ARM Cortex-M4 architecture, continuously advancing the state-of-the-art performance for HQC.

Outline of the Paper Section 2 establishes the algebraic notations and finite field background. Section 3 formalizes the Frobenius Additive FFT and introduces our Strided variant (SFAFFT). Section 4 details the SFAFFT-based polynomial multiplication and its application parameters for HQC. Finally, Section 5 and Section 6 present our platform-specific implementations and performance evaluations, respectively.

2 Preliminaries

This section fixes the algebraic notation used throughout the paper. We first recall Cantor bases and the additive-FFT CRT map, then summarize how these transforms are used for binary polynomial multiplication and for the HQC multiplication workload.

2.1 Binary Fields, Cantor Bases, and Additive FFT

Cantor Basis Let F be an extension of the binary field $\mathbb{F}_2$, and let $\{v_0 = 1, \dots, v_{m-1}\}$ be a basis of F . For a nonnegative integer t , define the field element $\underline{t} = \sum_{j=0}^{m-1} t_j v_j \in F$, where $t = t_0 + 2t_1 + 4t_2 + \dots$ is the binary representation of t . This basis also defines a tower of subspaces:

$$\begin{aligned} V_0 &= \{0\}, \\ V_1 &= \text{span}_{\mathbb{F}_2}\{v_0\} = \{0, 1\}, \\ V_2 &= \text{span}_{\mathbb{F}_2}\{v_0, v_1\} = \{0, 1, v_1, 1 + v_1\} \\ V_i &= \text{span}_{\mathbb{F}_2}\{v_0, v_1, \dots, v_{i-1}\} = \{0, 1, \underline{2}, \underline{3}, \dots, \underline{2^i - 1}\} \end{aligned}$$

Following the Cantor-basis construction [Can89, GM10], we term $\{v_0 = 1, \dots, v_{m-1}\}$ a Strong (we drop the “strong” below) Cantor basis if, for every integer i in $[1; m)$,

$$v_i^2 - v_i = v_{i-1}. \quad (1)$$

Subspace Polynomials We first define a family of polynomials over $F[x]$. Let $s(x) = x^2 - x$ and let

$$s_0 = x \quad s_i(x) = s(s_{i-1}(x)) = s_{i-1}(x)(s_{i-1}(x) - 1) \quad (2)$$

for positive integers i . If $\{v_0 = 1, v_1, \dots, v_{m-1}\}$ is a strong Cantor basis, then induction on j gives

$$s_j(v_i) = v_{i-j} \quad \text{for all } 0 \leq j \leq i < m.$$

We show that $s_i(x)$ is additive. Since F has characteristic 2, the Frobenius map is additive. Hence, for $x, y \in F$,

$$s(x + y) = (x + y)^2 + (x + y) = x^2 + y^2 + x + y = s(x) + s(y).$$

Inductively, we have $s_i(x + y) = s_i(x) + s_i(y)$.

We also record the factorization property of s_i , which is needed in the development of the ring-theoretic formalism of additive FFT. For $s(x)$, the construction immediately gives the factorization $x(x - 1)$. For $s_2(x)$, we have

$$s_2(x) = s_1(x)(s_1(x) - 1) = s_1(x)s_1(x - v_1) = x(x - 1)(x - v_1)(x - v_1 - 1),$$

because $s(v_1) = 1$. By manually computing small-degree s polynomials, one can see the following factorization template:

$$\begin{aligned} s_\ell(x - \beta) &= s_{\ell-1}(x - \beta)(s_{\ell-1}(x - \beta) - 1) \\ &= s_{\ell-1}(x - \beta)s_{\ell-1}(x - \beta - v_{\ell-1}). \end{aligned} \quad (3)$$

Here the second equality follows from $s_{\ell-1}(v_{\ell-1}) = 1$. Moreover, the two factors in (3) are coprime, since their difference is 1.

Repeated use of this template yields

$$\begin{aligned} s_\ell(x - \beta) &= \prod_{r \in \text{span}\{v_{\ell-1}\}} s_{\ell-1}(x - \beta - r) \\ &= \prod_{r \in \text{span}\{v_{\ell-2}, v_{\ell-1}\}} s_{\ell-2}(x - \beta - r) \\ &\dots \\ &= \prod_{r \in \text{span}\{v_0, \dots, v_{\ell-2}, v_{\ell-1}\}} s_0(x - \beta - r) = \prod_{r \in V_\ell} (x - \beta - r) \end{aligned}$$

From this, we also deduce that $s_\ell(x - \beta)$ is the vanishing polynomial of the coset $\beta + V_\ell$.

Additive FFT The additive fast Fourier transform (AFFT) [Can89, GM10] efficiently computes the values $[f(0), f(1), f(\underline{2}), f(\underline{3}), \dots, f(\underline{2^n - 1})]$ for a given polynomial $f \in \mathbb{F}_2[x] \subseteq F[x]$. Here, we restate this algorithm in ring-theoretic language. With the help of the Chinese Remainder Theorem, we can recover the algorithm without writing summation symbols, and this viewpoint will also enable the generalizations in Section 3.

The factorization in (3) gives the corresponding recursive Chinese Remainder Theorem decomposition:

$$\phi : \frac{F[x]}{\langle s_\ell(x - \beta) \rangle} \cong \frac{F[x]}{\langle s_{\ell-1}(x - \beta) \rangle} \times \frac{F[x]}{\langle s_{\ell-1}(x - \beta - v_{\ell-1}) \rangle}$$

Since $s_{\ell-1}(x - \beta) = s_{\ell-1}(x) - s_{\ell-1}(\beta)$ and $s_{\ell-1}(x - \beta - v_{\ell-1}) = s_{\ell-1}(x) - s_{\ell-1}(\beta) - 1$, this CRT map, given by natural projections, is effectively

$$\phi : f \mapsto (f \bmod (s_{\ell-1}(x) - s_{\ell-1}(\beta)), f \bmod (s_{\ell-1}(x) - s_{\ell-1}(\beta) - 1)).$$

We now show how to deduce the evaluation values $[f(0), f(1), f(\underline{2}), f(\underline{3}), \dots, f(\underline{2^n - 1})]$ from this CRT decomposition.

Take $[f(0), f(1), f(\underline{2}), f(\underline{3}), \dots, f(\underline{7})]$ as an example. We first map f into the quotient ring

$F[x]/\langle s_3(x) \rangle$. Then, according to the above CRT template, we have

$$\begin{aligned} \frac{F[x]}{\langle s_3(x) \rangle} &\stackrel{\text{CRT}_1}{\cong} \frac{F[x]}{\langle s_2(x) \rangle} \times \frac{F[x]}{\langle s_2(x - v_2) \rangle} \\ &\stackrel{\text{CRT}_2}{\cong} \frac{F[x]}{\langle s_1(x) \rangle} \times \frac{F[x]}{\langle s_1(x - v_1) \rangle} \times \frac{F[x]}{\langle s_1(x - v_2) \rangle} \times \frac{F[x]}{\langle s_1(x - v_2 - v_1) \rangle} \\ &\stackrel{\text{CRT}_3}{\cong} \frac{F[x]}{\langle x \rangle} \times \frac{F[x]}{\langle x - 1 \rangle} \times \frac{F[x]}{\langle x - v_1 \rangle} \times \frac{F[x]}{\langle x - v_1 - 1 \rangle} \\ &\quad \times \frac{F[x]}{\langle x - v_2 \rangle} \times \frac{F[x]}{\langle x - v_2 - 1 \rangle} \times \frac{F[x]}{\langle x - v_2 - v_1 \rangle} \times \frac{F[x]}{\langle x - v_2 - v_1 - 1 \rangle} \end{aligned}$$

The resulting sequence of natural projections yields the evaluation values $[f(0), f(1), f(2), \dots, f(7)]$ in the eight final subrings.

Generalizing this example, repeated use of the same CRT decomposition yields

$$\begin{aligned} \frac{F[x]}{\langle s_\ell(x - \beta) \rangle} &\cong \prod_{r \in \text{span}\{v_{\ell-1}\}} \frac{F[x]}{\langle s_{\ell-1}(x - \beta - r) \rangle} \\ &\cong \prod_{r \in \text{span}\{v_{\ell-2}, v_{\ell-1}\}} \frac{F[x]}{\langle s_{\ell-2}(x - \beta - r) \rangle} \\ &\dots \\ &\cong \prod_{r \in \text{span}\{v_0, \dots, v_{\ell-2}, v_{\ell-1}\}} \frac{F[x]}{\langle s_0(x - \beta - r) \rangle} = \prod_{r \in V_\ell} \frac{F[x]}{\langle x - \beta - r \rangle}. \end{aligned}$$

At the final stage, this gives the desired evaluation values on $\beta + V_\ell$.

Butterfly Algorithm In the CRT mappings above, one repeatedly computes reductions of the form

$$f \bmod (s_{\ell-1}(x) - \omega), \quad f \bmod (s_{\ell-1}(x) - \omega - 1),$$

where $\omega = s_{\ell-1}(\beta)$. If f is represented in the monomial basis $\{1, x, x^2, \dots\}$, these reductions require polynomial division by $s_{\ell-1}(x)$. This is not efficient for a recursive algorithm. Hence, we use the novel polynomial basis of Lin, Chung, and Han [LCH14], in which each CRT step becomes a butterfly operation.

Let n be a nonnegative integer, and write its binary representation as $n = n_0 + 2n_1 + 4n_2 + \dots$. Define

$$X_n = (s_0(x))^{n_0} (s_1(x))^{n_1} (s_2(x))^{n_2} \dots$$

The first few elements are: $X_0 = 1$, $X_1 = s_0(x)$, $X_2 = s_1(x)$, $X_3 = s_0(x)s_1(x)$.

Since the degree of $s_\ell(x)$ is 2^ℓ , we have $\deg X_n = n$, and the linear independence of $\{X_i\}$ follows.

The usefulness of this basis comes from the following observation. Suppose that, at some CRT step, we can write

$$f = A + s_{\ell-1}(x)B.$$

Then the two natural projections are

$$\begin{aligned} f \bmod (s_{\ell-1}(x) - \omega) &= A + \omega B, \\ f \bmod (s_{\ell-1}(x) - \omega - 1) &= A + (\omega + 1)B = A + \omega B + B. \end{aligned}$$

Thus each CRT step has the butterfly form

$$(A, B) \mapsto (A + \omega B, A + \omega B + B).$$

To illustrate the recursion, consider the second stage CRT_2 of the eight-point example. Let

$$a = [a_0, a_1, a_2, a_3], \quad b = [b_0, b_1, b_2, b_3]$$

be the two length-four novel-basis coefficient vectors produced by CRT_1 . The corresponding cosets have representatives 0 and v_2 , so their butterfly parameters are 0 and $\omega = s_1(v_2)$, respectively. Hence

$$\begin{aligned} \text{CRT}_2 : (a, b) \mapsto & ([a_0, a_1], [a_0 + a_2, a_1 + a_3], \\ & [b_0 + \omega b_2, b_1 + \omega b_3], [b_0 + (\omega + 1)b_2, b_1 + (\omega + 1)b_3]). \end{aligned}$$

Thus CRT_2 consists of two independent butterfly operations, and the subsequent stage applies the same rule recursively.

2.2 Polynomial Multiplication in $\mathbb{F}_2[x]$

The choice of polynomial multiplication algorithms in $\mathbb{F}_2[x]$ generally depends on the degree of the operands and the available hardware capabilities. We broadly categorize these into base, intermediate, and asymptotically fast algorithms.

For polynomials that fit within the machine word size, base multiplication relies directly on hardware-level carry-less instructions. As the degree scales beyond the base word size, implementations typically transition to divide-and-conquer methods, such as the Karatsuba or Toom–Cook algorithms [Knu97]. While Karatsuba reduces the computational complexity to $O(n^{\log_2 3})$ and serves as a highly efficient choice for medium-sized polynomials (e.g., in the official HQC implementation), these methods eventually become computational bottlenecks for very large degrees.

To achieve optimal scalability, Fast Fourier Transform (FFT) based algorithms are employed. Since traditional FFTs are not directly applicable over characteristic-two fields, specific variants tailored for binary fields have been developed, such as the Additive FFT (AFFT) [CCK⁺17] and the Frobenius Additive FFT (FAFFT) [LCK⁺18, CCK⁺18]. Specifically, computing a single polynomial multiplication via these methods consists of three main steps: two forward FFT operations to evaluate the input polynomials, one point-wise multiplication in the evaluation domain, and one inverse FFT operation to interpolate the final product. By executing these evaluation and interpolation steps efficiently, these FFT variants yield the lowest asymptotic complexity for large polynomial multiplication. The primary focus of this paper is the practical design and hardware-specific optimization of such FFT variants.

It should be noted that the practical crossover point—the threshold where FFT begins to outperform Karatsuba or Toom–Cook—is heavily influenced by the underlying architecture. For instance, processors featuring wide carry-less multipliers (such as the 64-bit PCLMULQDQ) extend the efficiency range of Karatsuba to higher degrees. Conversely, on architectures restricted to narrower multiplication instructions (e.g., the 8-bit PMULL.P8), the performance advantage of FFT emerges at much smaller polynomial lengths.

2.3 HQC

HQC (Hamming Quasi-Cyclic) [AAB⁺25] is a code-based post-quantum key encapsulation mechanism (KEM) recently selected by NIST for standardization [ACD⁺25]. As a KEM, it

provides three core cryptographic operations: key generation, encapsulation, and decapsulation. The primary computational bottleneck across these operations is the multiplication of large-degree polynomials in the ring $\mathbb{F}_2[x]/(x^n - 1)$. The concrete polynomial degrees are $n = 17669$, $n = 35851$, and $n = 57637$ for HQC-1, HQC-3, and HQC-5 respectively. This paper focuses on accelerating this specific operation to improve the overall performance of HQC.

2.4 Targeted Architectures

Our work targets a range of platforms, from high-end CPUs to edge and embedded systems, to demonstrate the broad applicability and architectural adaptability of our optimization techniques.

X86Avx2 The Intel AVX2 (Advanced Vector Extensions 2) instruction set provides sixteen 256-bit SIMD registers (ymm0-15). For polynomial multiplication in $\mathbb{F}_2[x]$, we leverage the PCLMULQDQ instruction from the PCLMULQDQ extension [Int14], which performs a $64 \times 64 \rightarrow 128$ -bit carry-less multiplication. We prioritize this standard instruction over the newer AVX-512 VPCLMULQDQ to ensure broader hardware compatibility across legacy systems.

GFNI Building upon the AVX2 ecosystem, the GFNI (Galois Field New Instructions) extension [Int21] significantly accelerates field arithmetic. By utilizing the GF2P8MULB instruction, we can execute parallel multiplications directly in $\mathbb{F}_{256} := \mathbb{F}_2[x]/(x^8 + x^4 + x^3 + x + 1)$. When vectorized with AVX2, this extension computes 32 such $\mathbb{F}_{256}$ multiplications simultaneously, drastically enhancing the throughput for byte-level polynomial arithmetic.

ApIM1 Representing high-performance ARMv8-A microarchitectures, the Apple M1 processor features the 128-bit NEON SIMD engine. It natively supports the PMULL.P64 instruction, which executes a $64 \times 64 \rightarrow 128$ -bit carry-less multiplication. This wide carry-less multiplier makes the platform highly efficient for algorithms explicitly optimized for large-word-size polynomial arithmetic.

ArmA72 The ARM Cortex-A72, commonly deployed in edge computing devices like the Raspberry Pi 4, provides a contrasting ARMv8-A environment. It heavily relies on the PMULL.P8 instruction, which performs eight parallel $8 \times 8 \rightarrow 16$ -bit carry-less multiplications. Optimizing for this narrower instruction is crucial for achieving high performance on edge platforms that lack efficient 64-bit carry-less hardware.

ArmM4 Serving as the NIST reference platform for embedded post-quantum cryptography benchmarking, the ARM Cortex-M4 [Arm20] implements the 32-bit ARMv7E-M ISA. Unlike the aforementioned platforms, it entirely lacks dedicated SIMD or carry-less multiplication instructions. Consequently, polynomial arithmetic relies on general-purpose integer instructions, such as the $32 \times 32 \rightarrow 64$ -bit multiply-and-accumulate (umlal), necessitating distinct, register-efficient algorithmic strategies to achieve optimal performance.

3 Strided Frobenius Additive Fast Fourier Transform

This section uses quotient-ring homomorphisms to formalize Frobenius additive FFT and the strided variant introduced below.

Throughout this section, we assume that m is a power of two and represent $K = \mathbb{F}_{2^m}$ using a Cantor basis $v_0, \dots, v_{m-1}$. For $0 \leq t \leq m$, write

$$V_t = \text{span}_{\mathbb{F}_2}\{v_0, \dots, v_{t-1}\}.$$

All affine spaces and Frobenius orbits below are taken with respect to this Cantor-basis representation.

3.1 Revisiting Frobenius Additive FFT

While the additive FFT (AFFT) [Can89, GM10, LCH14] provides an efficient evaluation algorithm for evaluation-and-interpolation-based multiplication, the Frobenius additive FFT (FAFFT) of Li et al. [LCK⁺18] further exploits Frobenius orbit structure in binary fields to recover additional evaluation values.

Let $\phi_2 : x \mapsto x^2$ be the Frobenius morphism in any extension of $\mathbb{F}_2$, and let $\text{Orb}_{\phi_2}(\alpha)$ denote the orbit of α under the iterates of ϕ_2 , i.e.,

$$\text{Orb}_{\phi_2}(\alpha) = \{\phi_2^j(\alpha) : j \geq 0\}.$$

Accordingly, the evaluation points in the FAFFT framework are organized into two sets. The full evaluation set Ω is a disjoint union of Frobenius orbits:

$$\Omega = \bigsqcup_{\sigma \in \Sigma} \text{Orb}_{\phi_2}(\sigma).$$

The set Σ is the corresponding cross section: each $\sigma \in \Sigma$ represents one Frobenius orbit contained in Ω , and distinct elements of Σ represent distinct orbits.

Orbits We first record the orbit structure used to choose the cross section Σ . Since the full evaluation set Ω is partitioned into Frobenius orbits, the size of Ω is determined by the number of representatives and the length of each orbit.

We use the following orbit-length property of Cantor bases [LCK⁺18, Lem. 3.1]. For every integer $t \geq 1$, the layer $v_t + V_t$ is stable under the Frobenius action. Moreover, every element in this layer has orbit length

$$|\text{Orb}_{\phi_2}(\alpha)| = 2^{\lceil \log_2 t \rceil + 1}, \quad \alpha \in v_t + V_t.$$

A direct computation of the first few layers gives

$$\begin{array}{ll} v_1 + V_1 = \text{Orb}_{\phi_2}(v_1), & |\text{Orb}_{\phi_2}(v_1)| = 2 \\ v_2 + V_2 = \text{Orb}_{\phi_2}(v_2), & |\text{Orb}_{\phi_2}(v_2)| = 4 \\ v_3 + V_3 = \text{Orb}_{\phi_2}(v_3) \sqcup \text{Orb}_{\phi_2}(v_3 + 1), & |\text{Orb}_{\phi_2}(v_3)| = 4 \\ v_4 + V_4 = \text{Orb}_{\phi_2}(v_4) \sqcup \text{Orb}_{\phi_2}(v_4 + v_1), & |\text{Orb}_{\phi_2}(v_4)| = 8 \\ v_5 + V_5 = \text{Orb}_{\phi_2}(v_5) \sqcup \text{Orb}_{\phi_2}(v_5 + 1) & |\text{Orb}_{\phi_2}(v_5)| = 8 \\ & \sqcup \text{Orb}_{\phi_2}(v_5 + v_2) \sqcup \text{Orb}_{\phi_2}(v_5 + v_2 + 1). \end{array}$$

For instance, to multiply two Boolean polynomials of length 4, one uses 8 evaluation points. Taking $\Sigma = \{v_3, v_3 + 1\}$ yields $\Omega = \text{Orb}_{\phi_2}(v_3) \sqcup \text{Orb}_{\phi_2}(v_3 + 1)$, which has size 8 and is therefore sufficient.

For length 8, one uses 16 evaluation points. The layer $v_4 + V_4$ already contains two orbits of length 8, with representatives $\{v_4, v_4 + v_1\}$. However, this set is not of the form $\alpha + V_\ell$, and hence it is not directly suitable for evaluation by AFFT. We therefore move to the next layer and take $\Sigma = \{v_5, v_5 + 1\} = v_5 + V_1$.

Following the affine Frobenius partition of Chen et al. [CCK⁺18, Eq. (6) and Props. 1–2], we take

$$\Sigma = v_{m/2+\ell} + V_\ell,$$

where m is a power of two and $0 \leq \ell < m/2$, so that $m/2 + \ell \leq m - 1$ and the element $v_{m/2+\ell}$ is defined in the chosen Cantor basis of $\mathbb{F}_{2^m}$. For this choice, each orbit has length m . We next verify that distinct elements of Σ represent distinct orbits. For $r \in V_\ell$, set $\sigma_r = v_{m/2+\ell} + r$. By the additivity of s_ℓ and $\ker(s_\ell) = V_\ell$, we have $s_\ell(\sigma_r) = v_{m/2}$. Suppose that $\sigma_{r'} = \phi_2^j(\sigma_r)$ for some $r, r' \in V_\ell$. Since s_ℓ commutes with Frobenius,

$$v_{m/2} = s_\ell(\sigma_{r'}) = s_\ell(\phi_2^j(\sigma_r)) = \phi_2^j(s_\ell(\sigma_r)) = \phi_2^j(v_{m/2}).$$

The orbit of $v_{m/2}$ has length m , so m divides j . Since ϕ_2^m is the identity on $\mathbb{F}_{2^m}$, it follows that $\sigma_{r'} = \sigma_r$ and hence $r' = r$. Therefore, these orbits are pairwise disjoint, and the corresponding full evaluation set has size

$$|\Omega| = |\Sigma| \cdot m = m2^\ell.$$

Evaluation on Orbit Representatives Once we observe that the values of an $\mathbb{F}_2$ -polynomial on Ω can be recovered from its values on Σ using Frobenius ϕ_2 , a natural multiplication algorithm suggests itself: evaluate the two operands on Σ , expand their values to Ω by repeated Frobenius ϕ_2 , multiply point-wise on Ω , and interpolate the product from Ω . As exploited by Chen et al. [CCK⁺18], this detour through Ω is unnecessary: the values on Σ alone already determine the product and suffice for interpolation. We next give a ring-theoretic formulation of this fact using the First Isomorphism Theorem [DF03, Sec. 7.3].

For the remainder of this subsection, fix $\Sigma = v_{m/2+\ell} + V_\ell$ and $K = \mathbb{F}_{2^m}$.

Proposition 1 (Evaluation on orbit representatives). *Let $0 \leq \ell < m/2$, and define*

$$q_{m,\ell}(x) = \mu_{v_{m/2}}(s_\ell(x)),$$

where $\mu_{v_{m/2}}(y) \in \mathbb{F}_2[y]$ is the minimal polynomial of $v_{m/2}$ over $\mathbb{F}_2$. Evaluation on $\Sigma = v_{m/2+\ell} + V_\ell$ induces an $\mathbb{F}_2$ -algebra isomorphism

$$\Psi_\Sigma : \frac{\mathbb{F}_2[x]}{\langle q_{m,\ell}(x) \rangle} \xrightarrow{\sim} \prod_{\sigma \in \Sigma} \frac{K[x]}{\langle x - \sigma \rangle}.$$

Proof. Start with evaluation on the full Frobenius-stable set:

$$\Phi_\Omega : \mathbb{F}_2[x] \longrightarrow \prod_{\omega \in \Omega} \frac{K[x]}{\langle x - \omega \rangle}, \quad f \longmapsto (f(\omega))_{\omega \in \Omega}.$$

For $\sigma \in \Sigma$, we have $s_\ell(\sigma) = v_{m/2}$. Since s_ℓ commutes with Frobenius,

$$s_\ell(\Omega) = s_\ell\left(\bigcup_{\sigma \in \Sigma} \text{Orb}_{\phi_2}(\sigma)\right) = \bigcup_{\sigma \in \Sigma} \text{Orb}_{\phi_2}(s_\ell(\sigma)) = \text{Orb}_{\phi_2}(v_{m/2}).$$

Hence $q_{m,\ell}$ vanishes on Ω . Moreover, $\deg q_{m,\ell} = \deg \mu_{v_{m/2}}(s_\ell(x)) = m2^\ell = |\Omega|$. Thus $q_{m,\ell}$ is the monic vanishing polynomial of Ω , and $\ker(\Phi_\Omega) = \langle q_{m,\ell} \rangle$.

By the First Isomorphism Theorem, Φ_Ω induces an isomorphism

$$\bar{\Phi}_\Omega : \frac{\mathbb{F}_2[x]}{\langle q_{m,\ell} \rangle} \xrightarrow{\sim} \text{im}(\Phi_\Omega).$$

It remains to show that $\text{im}(\Phi_\Omega)$ is already determined by the coordinates indexed by Σ . Let

$$\rho_\Sigma : \text{im}(\Phi_\Omega) \longrightarrow \prod_{\sigma \in \Sigma} \frac{K[x]}{\langle x - \sigma \rangle}$$

be coordinate restriction. If $\rho_\Sigma(\Phi_\Omega(f)) = 0$, then $f(\sigma) = 0$ for every $\sigma \in \Sigma$. For $\omega = \phi_2^j(\sigma) \in \Omega$, since $f \in \mathbb{F}_2[x]$, $f(\omega) = f(\phi_2^j(\sigma)) = \phi_2^j(f(\sigma)) = 0$. Thus $\Phi_\Omega(f) = 0$, so ρ_Σ is injective. Furthermore,

$$\dim_{\mathbb{F}_2} \text{im}(\Phi_\Omega) = \dim_{\mathbb{F}_2} \left(\frac{\mathbb{F}_2[x]}{\langle q_{m,\ell} \rangle} \right) = \deg q_{m,\ell} = m2^\ell = \dim_{\mathbb{F}_2} \left(\prod_{\sigma \in \Sigma} \frac{K[x]}{\langle x - \sigma \rangle} \right).$$

Therefore ρ_Σ is an isomorphism. The composition

$$\Psi_\Sigma = \rho_\Sigma \circ \overline{\Phi}_\Omega : \frac{\mathbb{F}_2[x]}{\langle q_{m,\ell} \rangle} \xrightarrow{\sim} \prod_{\sigma \in \Sigma} \frac{K[x]}{\langle x - \sigma \rangle}$$

is precisely evaluation on Σ , which proves the proposition. $\square$

Consequently, multiplication in the quotient can be carried out by evaluating both operands on Σ , multiplying their values point-wise, and interpolating the result via Ψ_Σ^{-1} . No evaluations on Ω need to be recovered.

Encode This realizes the Encode step of Chen et al. [CCK⁺18, Sec. 3.3, Alg. 4]. We next give a ring-theoretic interpretation of this map. The isomorphism Ψ_Σ can be realized as the following two steps:

$$\frac{\mathbb{F}_2[x]}{\langle q_{m,\ell}(x) \rangle} \xrightarrow[\sim]{\text{Enc}} \frac{\mathbb{F}_{2^m}[x]}{\langle s_\ell(x) - v_{m/2} \rangle} \xrightarrow[\sim]{\text{AFFT}} \prod_{\sigma \in \Sigma} \frac{\mathbb{F}_{2^m}[x]}{\langle x - \sigma \rangle}.$$

The first map is the natural reduction

$$\text{Enc}(f \bmod q_{m,\ell}(x)) = f \bmod (s_\ell(x) - v_{m/2}).$$

The following observation proves that this map is an isomorphism and will also be used for the strided construction.

Lemma 1 (Encode isomorphism). *Let $\beta \in K = \mathbb{F}_{2^m}$ have degree m over $\mathbb{F}_2$, and let $g(x) \in \mathbb{F}_2[x]$ be a monic polynomial of positive degree. Then the natural reduction map*

$$\text{Enc} : \frac{\mathbb{F}_2[x]}{\langle \mu_\beta(g(x)) \rangle} \longrightarrow \frac{K[x]}{\langle g(x) - \beta \rangle}, \quad f \bmod \mu_\beta(g(x)) \longmapsto f \bmod (g(x) - \beta),$$

is an $\mathbb{F}_2$ -algebra isomorphism.

Proof. The map is well-defined because $\mu_\beta(g(x))$ maps to $\mu_\beta(\beta) = 0$. Its image contains the residue class of x and also $\beta = g(x)$, and therefore contains $K = \mathbb{F}_2[\beta]$. Hence the map is surjective. Both algebras have dimension $m \deg g$ over $\mathbb{F}_2$, so the map is an isomorphism. $\square$

Taking $\beta = v_{m/2}$ and $g(x) = s_\ell(x)$ gives the first map above. As discussed in Section 2.1, reduction modulo $s_\ell(x) - v_{m/2}$ amounts to substituting $s_\ell(x) = v_{m/2}$. When f is represented in the novel basis, this substitution acts directly on its coefficients and can therefore be implemented efficiently. The second map evaluates the encoded polynomial on the affine space $s_\ell^{-1}(v_{m/2}) = v_{m/2+\ell} + V_\ell = \Sigma$, and is precisely an AFFT.

3.2 Strided Frobenius Additive FFT

Generalizing FAFFT The preceding subsection realizes FAFFT as the isomorphism

$$\frac{\mathbb{F}_2[x]}{\langle q_{m,\ell}(x) \rangle} \xrightarrow{\sim} \prod_{\sigma \in \Sigma} \frac{\mathbb{F}_{2^m}[x]}{\langle x - \sigma \rangle},$$

whose right-hand side consists of point evaluations on Σ . For polynomial multiplication, however, complete evaluation is unnecessary. If q is monic and $\deg q > \deg(fg)$, then multiplication modulo q recovers fg without wraparound. Thus only the degree of the quotient algebra on the left matters; a complete decomposition into evaluation components is not required.

This motivates us to consider the family

$$q_{m,\ell,k}(x) = \mu_\beta(s_\ell(x^k)) \quad \text{default } \beta = v_{m/2}, \quad Q_{m,\ell,k} = \frac{\mathbb{F}_2[x]}{\langle q_{m,\ell,k}(x) \rangle}. \quad (4)$$

Here m and k are powers of two, and ℓ is a nonnegative integer. This family gives two independent generalizations of FAFFT:

- The transform is *strided* if $k \geq 2$.
- The transform is *incomplete* if $\ell \geq m/2$, so that $s_\ell(z) - v_{m/2}$ does not split completely over $\mathbb{F}_{2^m}$.

Theorem 1 (SFAFFT decomposition). *Let $K = \mathbb{F}_{2^m}$ and let $Q_{m,\ell,k}$ be as in (4). Then Encode gives an $\mathbb{F}_2$ -algebra isomorphism*

$$Q_{m,\ell,k} \xrightarrow[\sim]{\text{Enc}} \frac{K[x]}{\langle s_\ell(x^k) - v_{m/2} \rangle}.$$

If $0 \leq \ell < m/2$, set $\alpha = v_{m/2+\ell}$, so that $s_\ell(\alpha) = v_{m/2}$. Then this isomorphism is followed by the CRT decomposition

$$Q_{m,\ell,k} \xrightarrow[\sim]{\text{Enc}} \frac{K[x]}{\langle s_\ell(x^k) - v_{m/2} \rangle} \xrightarrow[\sim]{\text{AFFT}} \prod_{r \in V_\ell} \frac{K[x]}{\langle x^k - \alpha - r \rangle}.$$

If $\ell \geq m/2$ and $d = \ell - m/2 + 1$, the corresponding incomplete decomposition is

$$Q_{m,\ell,k} \xrightarrow[\sim]{\text{Enc}} \frac{K[x]}{\langle s_\ell(x^k) - v_{m/2} \rangle} \xrightarrow[\sim]{\text{AFFT}} \prod_{\sigma \in v_{m-1} + V_{m/2-1}} \frac{K[x]}{\langle s_d(x^k) - \sigma \rangle}.$$

Proof. The Encode isomorphism follows from Lemma 1 with $\beta = v_{m/2}$ and $g(x) = s_\ell(x^k)$.

For the complete case, $s_\ell(\alpha) = v_{m/2}$. By additivity and the vanishing-polynomial identity for V_ℓ ,

$$s_\ell(z) - v_{m/2} = s_\ell(z - \alpha) = \prod_{r \in V_\ell} (z - \alpha - r).$$

Substituting $z = x^k$ gives the displayed factorization. The factors are pairwise coprime, since the difference of the factors indexed by distinct $r, r' \in V_\ell$ is the nonzero constant $r' - r$. The Chinese Remainder Theorem therefore gives the first decomposition.

For the incomplete case, set $h = m/2 - 1$, so that $d = \ell - h$. The recursive definition of the subspace polynomials gives $s_\ell = s_h \circ s_d$, while the strong Cantor-basis relation gives

$s_h(v_{m-1}) = v_{m/2}$. Hence

$$\begin{aligned} s_\ell(z) - v_{m/2} &= s_h(s_d(z) - v_{m-1}) \\ &= \prod_{r \in V_h} (s_d(z) - v_{m-1} - r). \end{aligned}$$

After substituting $z = x^k$, the factors are again pairwise coprime, because any two differ by a nonzero constant. Applying the Chinese Remainder Theorem proves the incomplete decomposition. Iterating the strong Cantor-basis relation shows that $s_d(K) = V_{m-d}$ when $d < m$, and $s_d(K) = \{0\}$ when $d \geq m$. Thus none of the elements in $v_{m-1} + V_{m/2-1}$ lies in $s_d(K)$, so these higher-degree factors have no roots in K . $\square$

When $\ell < m/2$, the first case gives $\Sigma = \alpha + V_\ell$. The same argument applies, with $q_{m,\ell,k}(x) = \mu_\beta(s_\ell(x^k))$, if $v_{m/2}$ is replaced by any degree- m element $\beta \in K$ for which $s_\ell(\alpha) = \beta$ for some $\alpha \in K$. When $\ell \geq m/2$, the second case stops at higher-degree factors. The conditions are independent; in particular, $k \geq 2$ and $\ell \geq m/2$ gives a strided incomplete FAFFT. Unlike ordinary FAFFT, the terminal components of SFAFFT need not be fields or linear evaluation factors.

Corollary 1 (Correctness of SFAFFT multiplication). *Let $f, g \in \mathbb{F}_2[x]$. If*

$$\deg(fg) < m2^\ell k = \deg q_{m,\ell,k},$$

then componentwise multiplication under either decomposition of Theorem 1, followed by the inverse isomorphism, recovers fg exactly.

Proof. The inverse isomorphism recovers the residue class of fg modulo $q_{m,\ell,k}$. Since fg already has degree smaller than the modulus, it is the unique representative of this class of degree less than $m2^\ell k$. $\square$

Benefit of SFAFFT Consider the 128-dimensional computation algebra used to multiply two length-64 Boolean polynomials. FAFFT takes $(m, \ell, k) = (16, 3, 1)$, whereas SFAFFT may take $(m, \ell, k) = (8, 3, 2)$:

$$Q_{16,3,1} \cong \prod_{\sigma \in v_{11} + V_3} \frac{\mathbb{F}_{2^{16}}[x]}{\langle x - \sigma \rangle} \quad \text{and} \quad Q_{8,3,2} \cong \prod_{\sigma \in v_7 + V_3} \frac{\mathbb{F}_{2^8}[x]}{\langle x^2 - \sigma \rangle}.$$

Both algebras have dimension 128 over $\mathbb{F}_2$. The additional parameter k therefore provides a tradeoff: while preserving the same no-wraparound precision, one may increase k and reduce m , moving the computation to a smaller extension field. In this example, SFAFFT replaces arithmetic in $\mathbb{F}_{2^{16}}$ with degree-two quotient arithmetic over $\mathbb{F}_{2^8}$; such smaller-field arithmetic generally requires fewer computational resources.

Relation to Incomplete NTT Readers familiar with polynomial-multiplication optimizations in lattice-based cryptography may recognize a connection to the so-called incomplete NTT [CHK⁺21, Sec. 2.4.4]. In our terminology, striding treats x^k as a new variable and applies the CRT decomposition until reaching factors that are linear in x^k , namely, factors of the form $x^k - \sigma$. For NTT, this is precisely where an incomplete transform stops. Thus strided NTT and incomplete NTT describe the same construction, whereas in FAFFT incompleteness may also arise from the nonsplitting of $s_\ell(x) - v_{m/2}$.

4 Strided FAFFT for Polynomial Multiplication in HQC

This section details the algorithmic parameters for accelerating polynomial multiplication in HQC. Our approach is presented in two main steps. First, we introduce the Strided Frobenius Additive FFT (SFAFFT), an optimization over the standard FAFFT for power-of-two length polynomials. We analyze its trade-offs and justify our choice of parameters. Second, we handle the non-power-of-two polynomial degrees specific to HQC by combining SFAFFT with the Chinese Remainder Theorem (CRT).

4.1 SFAFFT for Polynomial Multiplication

The Strided Frobenius Additive FFT (SFAFFT) is an optimization of the FAFFT algorithm that reduces the number of butterfly stages in the transform. The core idea is to reinterpret a polynomial in x as a polynomial in a new variable y , where $y = x^k$ and k is the stride length.

Consider a polynomial $f(x)$ of degree less than N where N is a power of two. By setting $y = x^k$, we can rewrite $f(x)$ as a polynomial $F(y)$ of degree less than N/k :

$$f(x) = \sum_{i=0}^{N-1} f_i x^i = \sum_{j=0}^{N/k-1} \left(\sum_{i=0}^{k-1} f_{j \cdot k + i} x^i \right) (x^k)^j = \sum_{j=0}^{N/k-1} F_j(x) y^j = F(y) \quad (5)$$

The coefficients $F_j(x)$ of this polynomial $F(y)$ are polynomials in x of degree less than k .

By applying FAFFT to $F(y)$ instead of $f(x)$, the length of the transform is reduced from N to N/k . This reduces the number of butterfly stages from $\log_2 N$ to $\log_2(N/k) = \log_2 N - \log_2 k$. From the perspective of the original data layout of $f(x)$, this is equivalent to performing the transform on coefficients that are k positions apart, hence the name ‘‘Strided FAFFT’’.

This reduction in butterfly stages comes at a cost: the point-wise multiplication step of the FFT is no longer a simple field multiplication. Instead, it becomes a multiplication of polynomials of degree less than k (the coefficients $F_j(x)$). This trade-off is beneficial when the savings from fewer butterfly stages outweigh the increased cost of point-wise polynomial multiplication, which can be efficiently handled by algorithms like Karatsuba.

4.1.1 The Stride-8 SFAFFT

To figure out the optimal stride length k , we evaluate the total number of $\mathbb{F}_{2^m}$ multiplications required to multiply two length-32 polynomials using SFAFFT with various strides. In our analysis, we assume that all point-wise multiplications in the evaluation domain are computed using the Karatsuba algorithm, and we also account for the multiplications required for the final ring reduction. As a baseline, a stride length of 32 represents a purely Karatsuba-based polynomial multiplication without any FFT stages. As demonstrated in Table 1, combining SFAFFT with Karatsuba yields the best performance at a stride length of 8, achieving the lowest overall multiplication count.

Moving from the standard FAFFT (stride-1) to the stride-8 SFAFFT modifies the four main operations over $\mathbb{F}_{2^m}$ as follows:

Basis Conversion A stride of 8 naturally aligns the operations with byte boundaries, allowing the basis conversion to be performed at the byte level rather than the bit level, which improves implementation efficiency.

Table 1: Comparison of $\mathbb{F}_{2^m}$ multiplication counts for multiplying two length-32 polynomials in $\mathbb{F}_{2^m}[x]/\langle x^{32} + \alpha \rangle$ where $\alpha \in \mathbb{F}_{2^m}$ using SFAFFT with various stride lengths.

Stride length k	32	16	8	4	2	1
# mul. in butterfly [†]	0	$3 \times 1 \times 16$	$3 \times 2 \times 16$	$3 \times 3 \times 16$	$3 \times 4 \times 16$	$3 \times 5 \times 16$
Point-wise mul. [‡]	$1 \times (243 + 31)$	$2 \times (81 + 15)$	$4 \times (27 + 7)$	$8 \times (9 + 3)$	$16 \times (3 + 1)$	32
Total $\mathbb{F}_{2^m}$ mul.	274	240	232	240	256	272

[†] Computed as $(\# \text{transforms}) \times (\# \text{stages}) \times (\# \mathbb{F}_{2^m} \text{ mul. per stage})$. Each multiplication requires three transforms (two forward, one inverse), each consisting of $\log_2(32/k)$ butterfly stages. Every stage contains 16 multiplications, independently of k .

[‡] Each entry is $(\# \text{point-wise products}) \times (\text{Karatsuba} + \text{ring reduction})$ multiplication counts. The $k = 32$ column corresponds to a purely Karatsuba-based multiplication without any butterfly stage.

Encode The algebraic structure is largely unchanged; the only modification is the data-extraction pattern used to build the polynomials, guided by the isomorphism:

$$\frac{\mathbb{F}_2[x]}{\langle \mu_\beta(s_\ell(x^8)) \rangle} \cong \frac{\mathbb{F}_2[x, y]}{\langle x^8 - y, \mu_\beta(s_\ell(y)) \rangle} \cong \frac{\mathbb{F}_{2^m}[x, y]}{\langle x^8 - y, s_\ell(y) - \beta \rangle}$$

The Butterfly Step By halting the recursive decomposition earlier, the stride-8 approach eliminates the final three butterfly stages of the standard FAFFT, reducing the total number of butterfly operations. The truncated decomposition is:

$$\frac{\mathbb{F}_{2^m}[x, y]}{\langle x^8 - y, s_\ell(y) - s_\ell(\alpha) \rangle} \cong \prod_{i=0}^{2^\ell-1} \frac{\mathbb{F}_{2^m}[x, y]}{\langle x^8 - y, y - \alpha - \underline{i} \rangle} \cong \prod_{i=0}^{2^\ell-1} \frac{\mathbb{F}_{2^m}[x]}{\langle x^8 - \alpha - \underline{i} \rangle}$$

Point-wise Multiplication Unlike the original FAFFT, which performs scalar $\mathbb{F}_{2^m}$ multiplications at the leaves of the recursion tree, the stride-8 SFAFFT requires polynomial multiplications within the quotient ring $\mathbb{F}_{2^m}[x]/\langle x^8 - \alpha - \underline{i} \rangle$. We handle these with the Karatsuba algorithm; although this stage costs more than its stride-1 counterpart, the trade-off is amortized by the savings in the butterfly stages (Table 1).

4.1.2 Choosing the Stride Length for the Target ISA

Beyond minimizing the multiplication count, the stride length k offers a second degree of freedom: it determines the field in which the butterfly arithmetic takes place, and we choose k so that this field is the smallest field that the target ISA multiplies efficiently.

On platforms with native 64-bit carry-less multiplication, a product in $\mathbb{F}_{2^{64}}$ already costs a single instruction, so enlarging k to shrink the field brings no benefit; we simply use the stride $k = 8$ that minimizes the multiplication count in Table 1. On byte-oriented platforms (**GFNI**, **ArmA72**) and on the bit-sliced Cortex-M4, the situation is reversed: field multiplication is the dominant cost, and its price grows quickly with the field size. We therefore adopt larger strides of 16 and 32 for HQC-1 and HQC-3, which bring the entire transform down to $\mathbb{F}_{2^{16}}$ —in contrast to previous FAFFT implementations operating over $\mathbb{F}_{2^{32}}$ [CCC⁺26, JLK⁺25]. For HQC-5 the trade-off is less clear-cut: a stride of 64 keeps the transform over $\mathbb{F}_{2^{16}}$ but turns the point-wise step into a costly length-64 Karatsuba multiplication, so we let the measured performance on each platform decide between a stride of 64 over $\mathbb{F}_{2^{16}}$ and a stride of 8 over $\mathbb{F}_{2^{32}}$. Although these larger strides exceed the multiplication-count optimum of Table 1, the smaller field reduces the cost of each multiplication by far more, yielding a net gain on these architectures.

To illustrate the effect of a larger stride, consider a stride-16 SFAFFT for polynomial multiplication in HQC-1. Its Encode and Butterfly steps follow exactly the same structure

as the stride-8 case above, now instantiated with $(m, \ell, k) = (16, 7, 16)$: Encode maps into the bivariate ring $\mathbb{F}_{2^{16}}[x, y]/\langle x^{16} - y, s_7(y) - \beta \rangle$, and, assuming $s_7(\alpha) = \beta$, the Butterfly step truncates the recursion into the $2^7 = 128$ components $\mathbb{F}_{2^{16}}[x]/\langle x^{16} - \alpha - \underline{i} \rangle$ for $0 \leq i < 128$.

For HQC-1 this reduces the butterfly decomposition to only 7 stages, so the additive FFT runs entirely over $\mathbb{F}_{2^{16}}$. When the transform can be kept over $\mathbb{F}_{2^{16}}$ rather than $\mathbb{F}_{2^{32}}$, this is especially beneficial on narrow-datapath architectures such as the Cortex-A72 (e.g., Raspberry Pi 4) and the Cortex-M4, which then use the available hardware far more efficiently.

4.2 Polynomial Multiplication in HQC

The main obstacle to fast polynomial multiplication in HQC is that the polynomial length n is not a power of two, which makes standard FFT-based algorithms inefficient. To address this, we combine the SFAFFT with the Chinese Remainder Theorem (CRT). Combining the CRT with additive-FFT variants for multiplying polynomials of length $n = 2^i + c$ in HQC was first explored in [CCPY26]; subsequent works [CCC⁺26, JLK⁺25] developed the FAFFT+CRT technique further.

CRT for lengths slightly above a power of two The CRT method targets products whose length is slightly above a power of two, i.e., $2^i + c$ with $c \ll 2^i$. Since $2^i = \deg q_{m, \ell, k} = m \cdot 2^\ell \cdot k$ and $\gcd(q_{m, \ell, k}, x^c) = 1$, the CRT yields the isomorphism

$$\mathbb{F}_2[x]_{< m 2^\ell k + c} \xleftrightarrow{\text{CRT}} \mathbb{F}_2[x]/\langle q_{m, \ell, k} \rangle \times \mathbb{F}_2[x]/\langle x^c \rangle, \quad (6)$$

where $q_{m, \ell, k}$ is a modulus polynomial for the ring of SFAFFT, and x^c handles the remaining part. The two component products are computed independently, and the final product is recovered through the inverse CRT map. Concretely, given $c_0 = f \bmod x^c$ and $c_1 = f \bmod q_{m, \ell, k}$, we seek the product $f \in \mathbb{F}_2[x]_{< 2^i + c}$ satisfying both congruences. The recovery proceeds in two steps:

1. Determine an auxiliary polynomial $u \in \mathbb{F}_2[x]_{< c}$ via

$$u \equiv (c_0 - c_1)/q_{m, \ell, k} \pmod{x^c}. \quad (7)$$

This step is computed either by multiplying $(c_0 - c_1)$ with a precomputed polynomial $I_q \equiv q_{m, \ell, k}^{-1} \pmod{x^c}$, or by dividing directly by $q_{m, \ell, k}$ modulo x^c , as described below.

2. Recover the product as

$$f = c_1 + u \cdot q_{m, \ell, k}. \quad (8)$$

Sparse modulus for SFAFFT in HQC For the $\mathbb{F}_{2^{16}}$ instantiations of Table 2, we have $m = 16$, $\ell = 7$, and $k = 16$ or 32 for HQC-1 and HQC-3, respectively. By (4), $q_{m, \ell, k} = \mu_\beta(s_\ell(x^k))$ leaves β as the only free parameter. An exhaustive search over all $\beta \in \mathbb{F}_{2^{16}}$ yields the 15-term modulus

$$\begin{aligned} q_{16, 7, k} = & x^{2048k} + x^{1792k} + x^{1537k} + x^{1282k} + x^{1027k} \\ & + x^{772k} + x^{517k} + x^{512k} + x^{262k} + x^{256k} + x^{8k} + x^{7k} + x^{2k} + x^k + 1. \end{aligned}$$

Compared with the 165- and 199-term moduli of [CCC⁺26], the SFAFFT modulus is considerably sparser, owing to the stride k .

For the $\mathbb{F}_{2^{64}}$ instantiations $(m, \ell, k) = (64, 6, 8)$ and $(64, 7, 8)$, we instead search over degree-64 elements $\beta \in s_\ell(\mathbb{F}_{2^{64}})$. A partial search already yields moduli with 35 and

29 nonzero terms for HQC-1 and HQC-3, respectively. These are denser than the $\mathbb{F}_{2^{16}}$ modulus, yet still far sparser than the 165- and 199-term moduli of [CCC⁺26].

Computing the auxiliary polynomial by division The sparsity of $q_{16,7,k}$ pays off once more in (7) for the $\mathbb{F}_{2^{16}}$ moduli: since $c \leq 256k$, only the five lowest-degree terms survive reduction modulo x^c ,

$$q_{16,7,k} \equiv 1 + x^k + x^{2k} + x^{7k} + x^{8k} \pmod{x^c},$$

and the inverse modulo x^c , which depends only on this residue, is nevertheless generically dense (about $c/2$ nonzero terms). Instead of multiplying by the precomputed inverse I_q , we solve $q_{16,7,k} \cdot u \equiv r \pmod{x^c}$ directly, where $r = c_0 - c_1$: since the constant term of $q_{16,7,k}$ is 1, comparing coefficients gives the linear recurrence

$$u_i = r_i + u_{i-k} + u_{i-2k} + u_{i-7k} + u_{i-8k}, \quad 0 \leq i < c, \quad (9)$$

with out-of-range terms taken as zero. A single low-to-high pass over (9) thus computes u with only four $\mathbb{F}_2$ -additions per coefficient — about $16\times$ fewer bit operations than multiplying by the dense I_q — although the recurrence is inherently sequential, so architectures with fast vectorized multipliers may still prefer the multiplication by I_q , as is also the case for the $\mathbb{F}_{2^{64}}$ moduli.

5 Implementation of Polynomial Multiplication in HQC

This section details our platform-specific implementations of polynomial multiplication for HQC on **X86Avx2**, **GFNI**, **ApIM1**, **ArmA72**, and **ArmM4**. Following (6), the polynomial multiplication for HQC-1 and HQC-3 is decomposed into two distinct quotient rings: $\mathbb{F}_2[x]/\langle q_{m,\ell,k} \rangle$, which is computed utilizing the SFAFFT, and $\mathbb{F}_2[x]/\langle x^c \rangle$, which handles the remainder. For HQC-5, the polynomial ring of the SFAFFT accommodates the full polynomial multiplication without the CRT decomposition.

5.1 SFAFFT for Polynomial Multiplication

As established in Section 4 and (5), the SFAFFT algorithm reinterprets an input polynomial $f(x)$ as a bivariate polynomial $F(y)$, where the coefficients $F_j(x)$ are polynomials of degree less than the stride length k . Because the SFAFFT applies the FAFFT over $F(y)$, it can be viewed as executing k independent FAFFT computations in parallel.

We exploit this inherent parallelism by mapping the k parallel transforms onto SIMD data lanes (for high-end processors) or into a bit-sliced architecture (for embedded platforms). Consequently, our SFAFFT implementations leverage the heavily optimized field arithmetic functions derived from state-of-the-art FAFFT implementations [CCPY26, CCC⁺26, JLK⁺25]. However, to fully utilize the k -way parallelism, we redesign the memory data layout and optimize the base field constructions (as detailed in Subsubsection 4.1.2).

Field Construction Our field constructions are tailored to each target ISA.

For high-end CPUs equipped with wide 64-bit carry-less multiplication instructions, namely **X86Avx2** and **ApIM1**, we construct the arithmetic over the field $\mathbb{F}_{2^{64}} := \mathbb{F}_2[x]/(x^{64} + x^4 + x^3 + x + 1)$. This construction perfectly maps to the PCLMULQDQ and PMULL.P64 instructions, ensuring maximum throughput for Galois Field operations.

On platforms optimized for 8-bit polynomial arithmetic, such as **GFNI** and **ArmA72**, we adopt a tower field construction starting from $\mathbb{F}_{2^{56}} := \mathbb{F}_2[x]/(x^8 + x^4 + x^3 + x + 1)$ (the standard 0x11b AES field). The SFAFFT is subsequently applied over the quadratic extension

$\mathbb{F}_{256^2} := \mathbb{F}_{256}[y]/(y^2 + y + x^5)$. Since $\mathbb{F}_{256^2} \cong \mathbb{F}_{2^{16}}$, this tower realizes the $\mathbb{F}_{2^{16}}$ arithmetic of [Subsubsection 4.1.2](#). This design is explicitly tailored for the **GF2P8MULB** instruction on **GFNI**, which natively accelerates $\mathbb{F}_{256}$ arithmetic. Similarly, on the **ArmA72**, this tower structure enables efficient arithmetic relying on the **PMULL.P8** instruction, which performs eight parallel $8 \times 8 \rightarrow 16$ -bit carry-less multiplications.

For the resource-constrained **ArmM4**, which lacks dedicated hardware for polynomial arithmetic, we utilize a fully bit-sliced implementation over a sequence of tower fields: $\mathbb{F}_{2^2} := \mathbb{F}_2[x_0]/(x_0^2 + x_0 + 1)$, $\mathbb{F}_{2^4} := \mathbb{F}_{2^2}[x_1]/(x_1^2 + x_1 + x_0)$, $\mathbb{F}_{2^8} := \mathbb{F}_{2^4}[x_2]/(x_2^2 + x_2 + x_1x_0)$, $\mathbb{F}_{2^{16}} := \mathbb{F}_{2^8}[x_3]/(x_3^2 + x_3 + x_2x_1x_0)$, and finally $\mathbb{F}_{2^{32}} := \mathbb{F}_{2^{16}}[x_4]/(x_4^2 + x_4 + x_3x_2x_1x_0)$. All field operations are emulated using bitwise logical instructions over the 32-bit general-purpose registers, allowing the processor to perform 32 field multiplications simultaneously via register-level SIMD.

Table 2 summarizes the underlying fields and target instructions for our SFAFFT implementations across the tested platforms.

Table 2: Summary of SFAFFT parameters (m, ℓ, k) across various platforms.

Platform	X86Avx2	AplM1	GFNI	ArmA72	ArmM4
Field	$\mathbb{F}_{2^{64}}$	$\mathbb{F}_{2^{64}}$	$\mathbb{F}_{256^2}$	$\mathbb{F}_{256^2}$	$\mathbb{F}_{2^{16}}, \mathbb{F}_{2^{32}}$
Instruction	PCLMULQDQ	PMULL.P64	GF2P8MULB	PMULL.P8	Bitwise Logic
HQC-1	(64, 6, 8)	(64, 6, 8)	(16, 7, 16)	(16, 7, 16)	(16, 7, 16)
HQC-3	(64, 7, 8)	(64, 7, 8)	(16, 7, 32)	(16, 7, 32)	(16, 7, 32)
HQC-5	(64, 8, 8)	(64, 8, 8)	(16, 7, 64)	(16, 7, 64)	(32, 9, 8)

Notes: Each entry gives the SFAFFT parameters (m, ℓ, k) of (4) and $\deg q_{m, \ell, k} = m \cdot 2^\ell \cdot k = 32768$, 65536, and 131072 for HQC-1, HQC-3, and HQC-5, respectively.

Butterfly Stages in SFAFFT During the butterfly stages, although the SFAFFT exposes k times more internal parallelism than the standard FAFFT, we maintain a computational topology similar to the FAFFT implementations of [\[CCPY26\]](#). We adapt several critical optimization techniques, such as merging multiple butterfly stages and delaying modular reductions (lazy reduction). These aggressive optimizations increase register pressure and are thus primarily deployed in our $\mathbb{F}_{2^{64}}$ implementations. Our SFAFFT implementations generally outperform the FAFFT because the increased parallelism yields a simpler, more regular memory access pattern, and the strided approach requires fewer butterfly stages for the same data size.

Point-wise Multiplication The point-wise multiplication of SFAFFT transitions from scalar Galois Field multiplications into polynomial multiplications of degree strictly less than k . We employ the Karatsuba algorithm to optimize the polynomial multiplication of small degrees.

Because there are 2^ℓ independent polynomial multiplications at this step, we reorganize the data layout to process multiple multiplications concurrently across different SIMD lanes. Through a data interleaving process similar to a matrix transpose, coefficients of the same polynomial degree from different polynomials are packed into the same SIMD register.

We also apply *lazy reduction* during this phase. Instead of performing a costly modular reduction after every sub-multiplication, we accumulate the intermediate products in wide registers as much as the data path allows, performing a single, batched reduction at the end. This is especially beneficial on architectures with a sufficient number of vector registers, such as **X86Avx2** and **AplM1**, where the risk of register spilling to the stack is minimal.

5.2 Optimizing CRT-based Multiplication

This section details the platform-specific optimizations for the CRT-based multiplication used to handle the non-power-of-two polynomial lengths in HQC-1 and HQC-3.

Multiplication in the Smaller Ring The multiplication within the smaller quotient rings, which are $\mathbb{F}_2[x]/\langle x^{3072} \rangle$ and $\mathbb{F}_2[x]/\langle x^{8192} \rangle$ in HQC-1 and HQC-3 respectively, reduces to a standard $\mathbb{F}_2[x]$ polynomial multiplication. We compute this using a truncated Karatsuba algorithm as in [CCC⁺26], strategically avoiding the calculation of high-degree terms that will simply be discarded by the modulo x^c reduction. The base case of this recursive algorithm heavily utilizes native $64 \times 64 \rightarrow 128$ -bit carry-less multiplication instructions, which are either directly supported (e.g., PCLMULQDQ and PMULL.P64) or efficiently emulated via parallel 8-bit instructions (PMULL.P8). On the **ArmM4**, which completely lacks native carry-less multiplication for $\mathbb{F}_2[x]$, we adopt a radix-16 polynomial multiplication approach utilizing the `umlal` instruction, mirroring the strategy in [CGKT22, CCC⁺26, JLK⁺25].

Inverse CRT Reconstruction The inverse CRT reconstruction is defined by (7) and (8).

For the sparse multiplication by $q_{m,\ell,k}$ in (8), we adopt different implementation strategies on high-end x86 (**X86Avx2** and **GFNI**) and Arm (**AplM1**) platforms. On the **AplM1**, we group the non-zero terms of the sparse polynomial into 64-bit blocks according to their degree. If a block contains multiple terms, we execute a carry-less multiplication (PMULL.P64) to process the multiplication by the entire block. If a block contains only a single term, the multiplication degenerates to a simple, fast XOR operation. On the **X86Avx2** and **GFNI**, we implement the entire sparse multiplication using XOR operations with proper casting, as processing the monomial contributions with 256-bit vector registers is more efficient than using PCLMULQDQ.

On the **ArmM4**, the sparse multiplication by $q_{m,\ell,k}$ is executed entirely through sequences of XOR operations. To alleviate the memory bandwidth bottleneck, we employ a *product scanning* technique for this sparse multiplication. Rather than repeatedly loading and storing partial sums to the destination buffer in memory, we iterate through the terms of the sparse multiplicand, computing their contributions and accumulating them directly within the CPU registers. A write-back to memory is only performed once a complete segment of the result is finalized.

Finally, when computing the auxiliary polynomial in (7), we again exploit the sparsity of the SFAFFT modulus $q_{m,\ell,k}$. On the **ArmM4**, we implement a direct division modulo x^c using the linear recurrence defined in (9). However, on high-end vector platforms equipped with fast native multiplication instructions, the data dependencies of the sequential recurrence make it less optimal. On these platforms, we instead compute this step by multiplying $(c_0 - c_1)$ with the precomputed dense inverse I_q . Even so, the extreme sparsity of the SFAFFT modulus remains a notable algorithmic improvement over prior FAFFT implementations [CCC⁺26].

5.3 Constant-Time Considerations

All implementations presented in this paper execute in constant time: the control flow and the sequence of memory addresses accessed are independent of secret data. This property follows from the structure of the algorithms. In the SFAFFT, all loop bounds, strides, and butterfly constants are fixed by the public parameters (m, ℓ, k) . The point-wise Karatsuba multiplications have a fixed recursion structure determined by the stride length k . In the CRT reconstruction, both the sparse multiplication by $q_{m,\ell,k}$ in (8) and the division in (9) are governed entirely by the public modulus.

At the instruction level, the carry-less and Galois-field multiplication instructions we employ (PCLMULQDQ, PMULL.P64, PMULL.P8, and GF2P8MULB) have data-independent latency on all tested microarchitectures. On the Cortex-M4, the UMLAL instruction executes in a single cycle regardless of its operands, and the remaining bit-sliced field arithmetic uses only constant-time bitwise logical instructions.

6 Experiment and Results

This section shows the SFAFFT performance of our implementations.

Experiment Setups We evaluated the performance of our implementation across five microarchitectures on four platforms: a high-end x86 server (AMD EPYC), an Apple M1 system, a low-power ARM embedded platform (Cortex-A72), and embedded systems (Arm Cortex-M4). The specifics of each platform and the build environment are detailed below.

- AMD EPYC 9754 (**X86Avx2** and **GFNI**): Benchmarks on the x86 platform were performed on an AMD EPYC 9754 CPU running Ubuntu 24.04.2 LTS. The code was compiled using GCC 13.3.0 with the `-O3 -mavx2 -mbmi -mpclmul` flags. The **GFNI** implementation was additionally compiled with the `-mgfni` flag. Core Performance Boost and simultaneous multithreading (SMT) were disabled. Cycle counts were measured using the RDTSC instruction.
- Apple M1 (**ApIM1**): Benchmarks were performed on an Apple M1 CPU running macOS 26.4. The code was compiled using Apple Clang 21.0.0 with the `-O3 -mcpu=apple-m1 -mtune=apple-m1` flags.
- ARM Cortex-A72 (**ArmA72**): Benchmarks were performed on a Raspberry Pi 4 featuring an Arm Cortex-A72 CPU and running Debian GNU/Linux 13. The code was compiled using GCC 14.2.0 with the `-O3 -march=native` flags. Cycle counts were measured using `perf_event_open` with the `PERF_COUNT_HW_CPU_CYCLES` event.
- ARM Cortex-M4 (**ArmM4**): Benchmarks were performed on the STM32L4R5 Nucleo-144 board. The board contains an STM32L4R5ZIT6 Arm Cortex-M4F microcontroller with 640 KB of SRAM and 2 MB of flash memory. We used the `pqm4` framework [KPR⁺], version `cc2c1b9`, with the `nucleo-l4r5zi` platform. All code was compiled using `arm-none-eabi-gcc 12.2.1` with the `-O3` flag.

All measurements report the median of 1000 measurements from random instances.

Source code The source code for all our implementations is included in the supplementary material of this submission and will be open-sourced under a CC0 copyright-waiver.

6.1 Performance on Polynomial Multiplication

Table 3 compares the cycle counts of FAFFT, Toom–Cook–Karatsuba, and our proposed SFAFFT. Across all tested platforms and parameter sets, SFAFFT outperforms the previous FAFFT implementations. The improvement is particularly significant for polynomial degrees of the form $2^k + c$, owing to our optimized sparse inverse CRT mapping.

On **X86Avx2** and **ApIM1**, the improvements mainly result from the reduced number of field multiplications discussed in Section 4. On the Raspberry Pi 4, Arm Cortex-M4, and **GFNI** platforms, SFAFFT additionally benefits from field representations better suited to the corresponding microarchitectures. The relative improvement becomes smaller for HQC-5, likely because the larger Karatsuba operands used in point-wise multiplication reduce the benefit of the proposed optimizations.

Table 3: Cycle counts of polynomial multiplication

Platform	Implementation	16 384	HQC-1 $n = 17\,669$	32 768	HQC-3 $n = 35\,851$	HQC-5 $n = 57\,637$
X86Avx2	[AAB ⁺ 25]	–	27 045	–	79 380	193 545
	[CCPY26]	43 720	52 987	92 345	123 480	200 655
	SFAFFT	39 506	47 318	86 447	109 237	192 893
	[CCPY26], GFNI	20 722	34 312	44 010	89 483	101 655
	SFAFFT, GFNI	15 952	22 770	36 833	59 197	91 463
ApIM1	[AAB ⁺ 25] ported	–	31 774	–	94 119	237 285
	[CCPY26]	28 468	37 008	59 975	93 080	130 050
	SFAFFT	24 948	29 124	54 812	70 885	117 115
ArmA72	[AAB ⁺ 25] ported	–	295 878	–	894 553	2 251 332
	[CCPY26]	300 066	338 316	631 173	848 087	1 348 167
	SFAFFT	159 148	194 727	372 632	498 681	814 516
ArmM4	[CCC ⁺ 26]	1 303 681	1 746 758	2 945 110	4 869 628	6 377 952
	[JLK ⁺ 25]	1 399 320	1 847 532	3 146 785	4 524 028	6 378 006
	SFAFFT	1 151 894	1 425 912	2 690 754	4 031 164	5 735 114

In [AAB⁺25], the original HQC implementation uses Toom–Cook–Karatsuba multiplication.

[CCPY26] combines FAFFT with Karatsuba multiplication on modern CPUs.

[CCC⁺26] and [JLK⁺25] use FAFFT+CRT multiplication on the Arm Cortex-M4.

Compared with Toom–Cook–Karatsuba, SFAFFT is faster for all HQC parameter sets on **ApIM1** and **ArmA72**. On **X86Avx2**, Toom–Cook–Karatsuba remains faster for HQC-1 and HQC-3, while the two implementations achieve nearly identical performance for HQC-5. This difference may be related to the latency and throughput of PCLMULQDQ on the tested AMD microarchitecture.

6.2 Performance Impact on HQC

Table 4 presents the comprehensive evaluation of the HQC scheme (encompassing Key Generation, Encryption, and Decryption) integrated with three distinct Boolean polynomial multiplication paradigms. To further optimize our implementation, we incorporate the common input transform technique originally proposed in [CCPY26]. While this optimization is mathematically applicable to both algorithms, SFAFFT inherently derives a smaller marginal benefit from it. As analyzed in Section 4, this profile stems from the fact that SFAFFT fundamentally shifts the computational bottleneck, spending a higher relative proportion of cycles on core point-wise multiplications rather than on the forward and inverse polynomial transforms. Overall, SFAFFT provides substantial performance improvements in most evaluated settings, but it is not uniformly faster. In particular, for HQC-5 encapsulation and decapsulation with **GFNI**, SFAFFT performs essentially on par with FAFFT (within 1.5%), consistent with the computational trade-off discussed above.

6.3 Conclusion

Our contributions span both theoretical foundations and practical implementations. The introduction of the ring isomorphism perspective formalizes the algebraic structure of FAFFT, which not only deepens the theoretical understanding of additive FFT variants but also directly facilitates the discovery of Strided-FAFFT (SFAFFT).

On the practical side, the proposed SFAFFT demonstrates high efficiency when integrated into the multiplication component of the HQC cryptosystem. By tailoring optimization

Table 4: Kilo-cycle counts of HQC implementations

Platform	Implementation	HQC-1	HQC-3	HQC-5
		Key/Enc/Dec	Key/Enc/Dec	Key/Enc/Dec
X86Avx2	[AAB ⁺ 25]	69 / 140 / 311	170 / 337 / 667	353 / 695 / 1347
	[CCPY26]	95 / 174 / 345	212 / 390 / 704	361 / 646 / 1207
	SFAFFT	91 / 170 / 337	199 / 372 / 674	348 / 639 / 1184
	[CCPY26], GFNI	77 / 144 / 295	180 / 343 / 625	258 / 461 / 907
	SFAFFT, GFNI	67 / 127 / 268	149 / 287 / 538	254 / 468 / 908
AplM1	[AAB ⁺ 25] ported	75 / 154 / 309	194 / 392 / 675	422 / 836 / 1408
	[CCPY26]	78 / 157 / 313	188 / 390 / 642	303 / 577 / 1021
	SFAFFT	72 / 141 / 291	173 / 325 / 574	301 / 558 / 994
ArmA72	[AAB ⁺ 25] ported	415 / 806 / 1371	1174 / 2304 / 3665	2760 / 5436 / 8541
	[CCPY26]	456 / 802 / 1394	1099 / 1949 / 3188	1850 / 3256 / 5384
	SFAFFT	314 / 564 / 1019	774 / 1435 / 2375	1337 / 2503 / 4126
ArmM4	[CCC ⁺ 26]	2901 / 5109 / 8642	7976 / 14283 / 22684	12577 / 21689 / 34543
	[JLK ⁺ 25]	2992 / 5771 / 9886	7612 / 14602 / 23180	12577 / 23933 / 38344
	SFAFFT	2580 / 4699 / 7910	7137 / 13265 / 20827	11981 / 21225 / 33483

strategies to various hardware constraints, our implementation delivers consistent speedups over existing FFAFFT approaches.

References

- [AAB⁺25] Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaiieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, Jurjen Bos, Arnaud Dion, Jerome Lacan, Jean-Marc Robert, Pascal Veron, Paulo L. Barreto, Santosh Ghosh, Shay Gueron, Tim Güneysu, Rafael Misoczki, Jan Richter-Brokmann, Nicolas Sendrier, Jean-Pierre Tillich, and Valentin Vasseur. HQC. Technical report, pqc-hqc.org, 2025. available at https://pqc-hqc.org/doc/hqc_specifications_2025_08_22.pdf.
- [ACD⁺25] Gorjan Alagic, David Cooper, Quynh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, and Daniel Smith-Tone. Status report on the fourth round of the NIST post-quantum cryptography standardization process. Technical Report NIST IR 8545, National Institute of Standards and Technology (NIST), March 2025.
- [Arm20] Arm Limited, Cambridge, UK. *Arm Cortex-M4 Processor Technical Reference Manual*, issue 1.0 edition, September 2020. Document number 100166_0001_04_en, available at <https://documentation-service.arm.com/static/5fce431be167456a35b36ade>.
- [Can89] David G Cantor. On arithmetical algorithms over finite fields. *Journal of Combinatorial Theory, Series A*, 50(2):285–300, 1989.
- [CCC⁺26] Ming-Shing Chen, Tun-You Chien, Chun-Ming Chiu, Han-Hsuan Lin, Chun-Tao Peng, and Bo-Yin Yang. Additive FFTs for HQC on ARM cortex-m4, revisited. Cryptology ePrint Archive, Paper 2026/739, 2026. To appear in IACR TCHES 2026(3).
- [CCK⁺17] Ming-Shing Chen, Chen-Mou Cheng, Po-Chun Kuo, Wen-Ding Li, and Bo-Yin Yang. Faster multiplication for long binary polynomials. *CoRR*, abs/1708.09746, 2017.

- [CCK⁺18] Ming-Shing Chen, Chen-Mou Cheng, Po-Chun Kuo, Wen-Ding Li, and Bo-Yin Yang. Multiplying boolean polynomials with frobenius partitions in additive fast fourier transform. *CoRR*, abs/1803.11301, 2018.
- [CCK21] Ming-Shing Chen, Tung Chou, and Markus Krausz. Optimizing BIKE for the intel haswell and ARM Cortex-M4. *IACR TCHES*, 2021(3):97–124, 2021.
- [CCPY26] Ming-Shing Chen, Chun-Ming Chiu, Chun-Tao Peng, and Bo-Yin Yang. Accelerating HQC with additive FFT. Cryptology ePrint Archive, Report 2026/14, 2026. To appear in IACR TCHES 2026(2).
- [CGKT22] Ming-Shing Chen, Tim Güneysu, Markus Krausz, and Jan Philipp Thoma. Carry-less to BIKE faster. In Giuseppe Ateniese and Daniele Venturi, editors, *ACNS 2022*, volume 13269 of *LNCS*, pages 833–852, Rome, Italy, June 20–23, 2022. Springer, Cham, Switzerland.
- [CHK⁺21] Chi-Ming Marvin Chung, Vincent Hwang, Matthias J. Kannwischer, Gregor Seiler, Cheng-Jih Shih, and Bo-Yin Yang. NTT multiplication for NTT-unfriendly rings: New speed records for Saber and NTRU on Cortex-M4 and AVX2. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021(2):159–188, 2021.
- [DF03] D.S. Dummit and R.M. Foote. *Abstract Algebra*. Wiley, 2003.
- [GM10] Shuhong Gao and Todd D. Mateer. Additive fast fourier transforms over finite fields. *IEEE Trans. Inf. Theory*, 56(12):6265–6272, 2010.
- [Int14] Intel. Intel carry-less multiplication instruction and its usage for computing the gcm mode, 2014. <https://cdrdv2-public.intel.com/836172/clmul-wp-rev-2-02-2014-04-20.pdf>.
- [Int21] Intel. Galois field new instructions (gfni) technology guide, 2021. https://cdrdv2.intel.com/v1/dl/getContent/644497?fileName=GFNI_TechGuide_644497v2.pdf.
- [JLK⁺25] Jihoon Jang, Myeonghoon Lee, Donggeun Kwon, Seokhie Hong, Suhri Kim, and Sangjin Lee. Efficient polynomial multiplication for HQC on ARM cortex-m4. Cryptology ePrint Archive, Paper 2025/1939, 2025. Refer to the 2026-01-17 version.
- [Knu97] Donald E. Knuth. *The art of computer programming, volume 2 (3rd ed.): seminumerical algorithms*. Addison-Wesley Longman Publishing Co., Inc., USA, 1997.
- [KPR⁺] Matthias J. Kannwischer, Richard Petri, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. PQM4: Post-quantum crypto library for the ARM Cortex-M4. <https://github.com/mupq/pqm4>.
- [LANHC16] Sian-Jheng Lin, Tareq Y. Al-Naffouri, Yung-Hsiang S. Han, and Wei-Ho Chung. Novel polynomial basis with fast fourier transform and its application to reed-solomon erasure codes. *IEEE Transactions on Information Theory*, 62(11):6284–6299, 2016.
- [LCH14] Sian-Jheng Lin, Wei-Ho Chung, and Yung-Hsiang S. Han. Novel polynomial basis and its application to Reed-Solomon erasure codes. In *55th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2014, Philadelphia, PA, USA, October 18-21, 2014*, pages 316–325. IEEE Computer Society, 2014.

- [LCK⁺18] Wen-Ding Li, Ming-Shing Chen, Po-Chun Kuo, Chen-Mou Cheng, and Bo-Yin Yang. Frobenius additive fast fourier transform. In Manuel Kauers, Alexey Ovchinnikov, and Éric Schost, editors, *Proceedings of the 2018 ACM on International Symposium on Symbolic and Algebraic Computation, ISSAC 2018, New York, NY, USA, July 16-19, 2018*, pages 263–270. ACM, 2018.
- [LJKH25] Myeonghoon Lee, Jihoon Jang, Suhri Kim, and Seokhie Hong. Improved frobenius fft for code-based cryptography on cortex-m4. *IEEE Internet of Things Journal*, 12(17):36318–36327, 2025.
- [RLC⁺25a] Antonio Ras, Antoine Loiseau, Mikael Carmona, Simon Pontié, Guénaél Renault, Benjamin Smith, and Emanuele Valea. Optimizing hqc using frobenius additive fft on a risc-v-based system-on-chip. NIST Sixth PQC Standardization Conference, 2025. available at <https://csrc.nist.gov/csrc/media/events/2025/sixth-pqc-standardization-conference/optimizing%20hqc%20using%20frobenius%20additive%20fft.pdf>.
- [RLC⁺25b] Antonio Ras, Antoine Loiseau, Mikael Carmona, Simon Pontié, Guénaél Renault, Benjamin Smith, and Emanuele Valea. PHOENIX: Crypto-agile hardware sharing for ML-KEM and HQC. Cryptology ePrint Archive, Report 2025/601, 2025.
- [vdHL17] Joris van der Hoeven and Robin Larrieu. The frobenius fft. In *Proceedings of the 2017 ACM International Symposium on Symbolic and Algebraic Computation, ISSAC '17*, page 437–444, New York, NY, USA, 2017. Association for Computing Machinery.